Lore

The engine behind a greenfield build: agents do the work, people decide. Shown on real runs, not slides.

01
ai-agent-subsystem
one agent run = one Kubernetes resource

02
Floor
assembly lines · runs · the queue

03
Lore
GitHub · stations · context · people

04
Knowledge graph
what the org knows, with its source

Who is in the call

re:cinq

- Michael Mueller · CTO, co-founder of re:cinq
- Bogdan Szabo · engineer on Lore, leads the walkthrough
- Loredana Moanga · automated merging and the reviewer app

Intro · agenda

Four blocks, mostly live

We only show material that is public or anonymised. Pricing, staffing and tool costs are not part of this call; the RFP covers them.

1 · 35–40 MIN

Evidence walkthrough

Real runs from work item through agent run, tests and CI, review and merge, and a corrected case: what the machine does when something fails.

2 · ~20 MIN

Control model

How human review is enforced and recorded, who may apply the automerge label, how gate files are protected, where model and MCP data flows.

3 · ~15 MIN

Target and team

Azure Pipelines as the CI/CD standard, PostgreSQL RLS and tenant recovery, the people on both sides.

4 · ~10 MIN

Close

Organisational repeatability and the process after this call.

Intro · why we are here

We show the engine, not a product

THE IDEA

We don't build the system for you. We build the system that builds it.

Agents do the work, people decide, and your teams run the engine.

YOUR GOAL

A clean greenfield build

A full re-implementation next to the existing platform.

OUR APPROACH

An engine your teams run

Plan, spec, tickets, code, CI and review as assembly lines, with people at the decisions that matter.

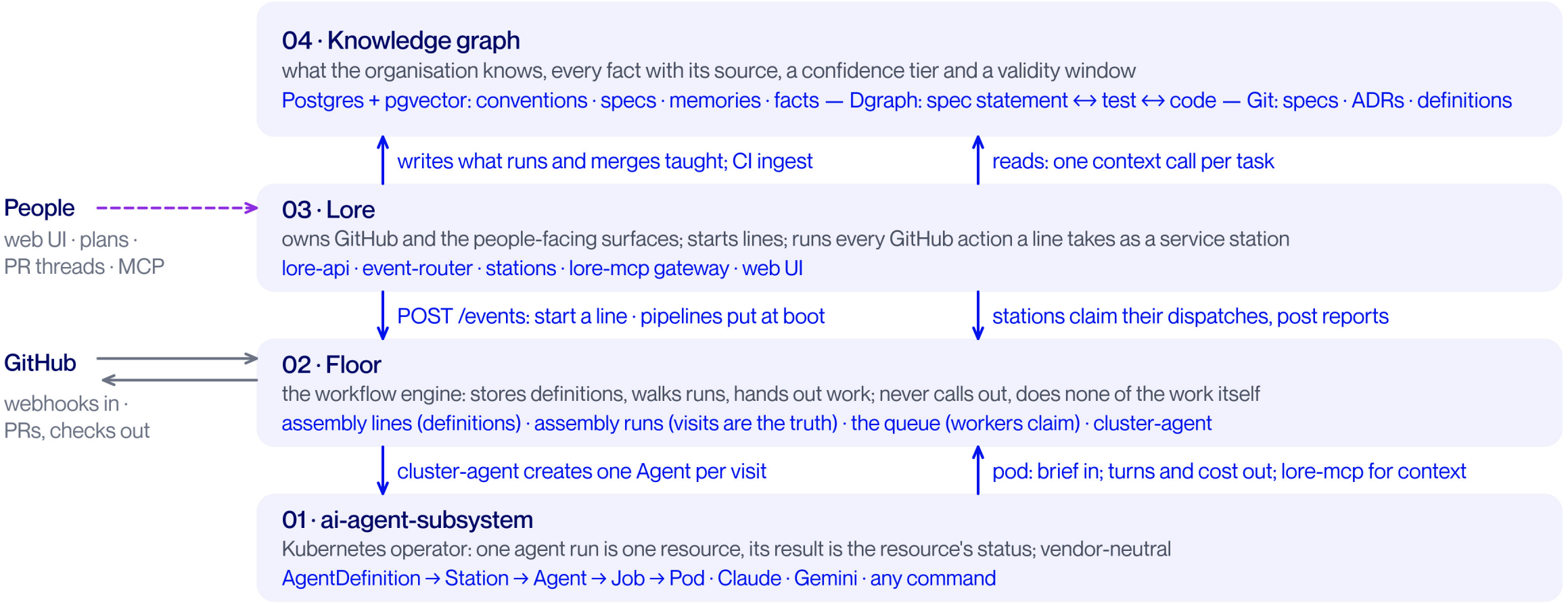
TODAY

Lore as the working example

Our own implementation of that engine, shown on our repositories and real pull requests.

Four layers, each with one job

Every blue arrow is an HTTP call made by the layer that needs something: nothing pushes into a pod or into the floor. Violet is where people act.



Intro · the stages you'll see

From plan to merge, and where people decide

Each stage is prepared in advance, because one step takes 5–10 minutes live. We show the state at every stage, then run one through.



plan	spec	tickets	implement	CI	review	merge
People and the agent edit the plan; a person approves it	Written by the agent, reviewed and merged by a person	The merged spec, split into tickets sized for one run	Red tests first, then code, in a fresh pod per visit	CI judges every push; the line waits at no cost	Findings posted inline; people answer the threads	A person merges, unless lights-out allows low-risk paths

violet = a person decides · blue = agent station · dashed = parked, no pod

human — People decide at three points: approving the plan, merging the spec, and merging the code. The factory never changes its own rules.

1 Evidence walkthrough · 35–40 min · live

Live: real runs, from work item to merge

We follow each run on GitHub and in Lore, stage by stage, and answer questions as we go.

#2064

Work item to merge

Work item, agent run, tests and CI, review, merge.

#2065

A second run, end to end

The same path on a different change.

corrected

When something fails

What the machine does on failure: the red build, the repair or hand-back, and the eventual merge.

ON EVERY RUN the frozen brief, station and agent definition pinned by content hash, every CI verdict, the review threads, and the cost.

1 Evidence walkthrough · live · #2064

#2064: work item to merge, by the factory

A person filed an outage ticket. The implementation line wrote the failing check, the fix and the pull request in 32 minutes. A person merged it. Every link below is the production record.

1 · WORK ITEM

Ticket #1948

Filed by a person on 9 Sep after a migration hook failed a deploy. Picked up by the backlog loop on 13 Sep at 21:38 UTC.

Issue #1948

"Agent picked up this task"

2 · AGENT RUN

Seven visits, one run

`dod → open-pr → tdd-round → await-ci → ready-for-review → await-pr → retrospective`

21:38 to 22:10. Red test first, then the fix, each in its own pod.

Run page in Lore `dbccc0a0`

Task page `27b3ec3b`

Red test · the fix · clean-up

3 · TESTS AND CI

26 checks green

CI judged every push. The `apply` job runs the acceptance check the agent wrote: migrations as the application role against the full baseline.

All checks on the PR

migrations · apply

integration

4 · REVIEW

By the person who merged

On 13 Sep the reviewer did not yet judge pull requests the factory itself opened. Since 25 Sep its verdict lands on a `lore/code-review` check.

PR #2064, description and diff

#2196: verdict on factory PRs

5 · MERGE

A person, 22:56

Squash-merged by a maintainer. The PR body carries `Lore-Task`, the commits carry the model that wrote them.

Merge commit 41fbfcf87

Commits and trailers

IN LORE The run page still holds the full transcript: 518 recorded agent turns across the seven visits, with every tool call and the cost.

1 Evidence walkthrough · live · #2065

#2065: a person's change, corrected by the reviewer

The same evening, a maintainer opened a pull request by hand. Lore's reviewer blocked it five minutes later for a real defect, the author fixed it, and every later push was re-checked.

1 · WORK ITEM

A human pull request

Opened 13 Sep at 21:52 UTC: a loop ticket that failed for infrastructure reasons is deferred, a bounded number of times.

PR #2065

#1648 · #2056 the tickets

2 · REVIEW RUN

Blocked at +5 minutes

The code-review line read the change in a clone and posted CHANGES_REQUESTED: one must-fix, one nit. The review names its model and run.

The review, 21:57

Run page in Lore 9a27aff3

3 · THE FINDING

A real double-count

The closing run was already marked failed, so the first failure counted as attempt two. Posted on the line, with a suggested fix.

The blocking finding

The author's answer

A nit, declined with a reason

4 · FIX AND RE-CHECKS

Approved on every push

Fixed in one commit with a new test. Four re-check runs, each judging only what changed since the last verdict.

The fix, 016536120

First re-check, 22:02

Last re-check, 22:34

A re-check run in Lore

5 · CI AND MERGE

Green, merged at 22:39

All checks green, including `lore/code-review`. The author merged. The declined nit was taken two PRs later.

Checks on the PR

#2067, where the nit landed

WHAT IT SHOWS Review is the same line for a person's pull request and for the factory's. People answer in the thread; the answer, and the reason, stay on the record.

1 Evidence walkthrough · live · when something fails

When something fails: three real cases

Retries are bounded and named in the line definition. When a bound is spent the factory stops, says why on the ticket, and leaves the pull request for a person.

#2053

A red build, repaired, then merged

CI went red after the first round, and stayed red after the second. The repair station took CI's failing checks, fixed only those, and the build went green.

`tdd-round → CI red → tdd-round → CI red → repair-build → CI green → ready-for-review → merged`

Run page in Lore `7217c809 · 11 visits · 730 turns`
The red check · the repair commit
PR #2053 · ticket #2025

#2060

Out of repairs: handed to a person

The build was still red after three fixes. The loop spent its budget, labeled the ticket `lore:blocked`, and wrote the reason on it. The pull request stayed open for a human; it was closed unmerged.

`await-pr → fix-ci ×3 → parked`

The parking comment
Run page in Lore `d72bb72f`
PR #2060 · ticket #1899

#1846

A ticket it could not build: declined in six minutes

The ticket asked for a lint rule, but that rule's code lives in another repository. The first station saw it could not write a failing test here. It wrote no code, opened no pull request, and parked the ticket with the reason and where to re-file it.

`dod → parked`

The parking comment, with the reason
Run page in Lore `904278cf · 131 turns`
Ticket #1846

THE BOUNDS 12 rounds per ticket · 2 build repairs during the rounds · 3 CI fixes before review · 1 retry of a failed step. A spent bound is a finding for planning, not a silent loop.

Human review: enforced by structure, recorded on the run

Enforced

- Human nodes dispatch nothing and wait with no deadline: approving the plan, merging the spec.
- The implementation line never merges. A person does, unless lights-out is on for low-risk paths.
- The factory reads its rules and never writes them: line and agent definitions, trust levels, allowlists, lights-out.
- Privileged settings take two people.

Recorded

- Every visit pins its station and agent definition by content hash.
- Status, current node and cost are replayed from the visits, so a run can always be explained.
- Reviews, findings and re-checks live on the pull request.

`human` In the demo: approve a plan, request changes to stop the line on a PR, and take over a ticket labeled lore:blocked.

Who may apply the automerge label

TODAY

Merge is a person's call

Lights-out, set by people through the settings and never by the factory, allows automerge for low-risk paths only.

THE CONSTRAINT

One account cannot do both

GitHub does not let the same account approve and merge a pull request.

IN PROGRESS

A separate reviewer app

A reviewer app, inside or next to Lore, acting as a second GitHub identity for approvals and merges.

How workflow and gate files are protected

What the design guarantees

- Line definitions, agent definitions, trust levels, allowlists and lights-out are read by the factory, never written.
- They change only by pull request or the settings page.
- Privileged settings take two people.
- A pod gets git credentials only by asking the floor; Lore holds the GitHub credentials.

Claude, Gemini and MCP: where data goes

What the design guarantees

- The agent run is vendor-neutral: Claude, Gemini or any command.
- The pod calls out to the model vendor; nothing calls into the pod.
- Agents reach Lore through lore-mcp for context and memory, one context call per task.
- The knowledge graph stores no secrets: every write is redacted, and nothing lands without a source.

Azure Pipelines as the CI/CD standard

Why the harness fits any CI

- The line never runs lint, typecheck or the suite itself. CI does, on every push.
- The line parks while CI runs, with no pod and no cost.
- The pull request's own build decides the next node: green moves on, red goes to the next round, a repair, or a fix from CI's report.
- Agents read the CI verdict through lore-mcp.

PostgreSQL RLS and single-tenant recovery

SINGLE-TENANT

One tenant per database

Lore runs on the database it is given through a connection string. Run for you, it would get a dedicated instance.

RLS

Nothing to partition

Row-level security separates tenants that share one database. With one tenant per database, that boundary is the database itself.

ACCESS

No per-user roles in Lore

Lore works within the access it is granted. Who may do what is set where that access is given.

4 Close · ~10 min

Repeatable beyond one team, and what comes next

Repeatability

- We provide the engine; your teams run it.
- Line and agent definitions are files, changed by pull request, so a new team starts from reviewed definitions rather than from scratch.

Pricing, staffing and tool costs are not covered here; the RFP handles them.